

Bifurcation - a walkthrough

Bifurcation

Two hundred and fifty lines that make one argument: compute in Mojo, present in Python, and measure the boundary rather than asserting it.

It sweeps the logistic map $x \rightarrow r x (1 - x)$ across 1,600 values of r . The attractor doubles, doubles again, and falls into chaos — with windows of order inside it. The lower panel is the Lyapunov exponent, and it dips below zero exactly where those windows are.

Source	examples/bifurcation/main.mojo, 250 lines, one file
Grid	1,600 columns of $r \times 900$ buckets of x
Work	6,400,000 iterations, each depending on the one before it
Mojo	23.8 ms for all 1,600 columns
CPython	8.5 ms for 24 columns $\rightarrow$ 0.6 s extrapolated
Ratio	20.2x , measured on the same algorithm both sides

These documents

Chapter	What it covers
1. The map	Period doubling, chaos, the windows of order, and what the lower panel measures
2. The division of labour	Why the arithmetic stays in Mojo, why the figure goes to Python, and how the 20x was measured honestly
3. Crossing the bridge	Copying arrays wholesale, sending CPython its own source, and two plotting details worth stealing
4. What to understand	The things that will surprise you, and the ones that will bite

The shortest possible summary

The thing Python is very good at is not arithmetic. It is everything that surrounds a result: axes with sensible ticks, a colour map with a legend, a norm that makes a sparse histogram readable, two panels sharing an x-axis, and a PNG at the end. Those are thousands of decisions somebody has already made well.

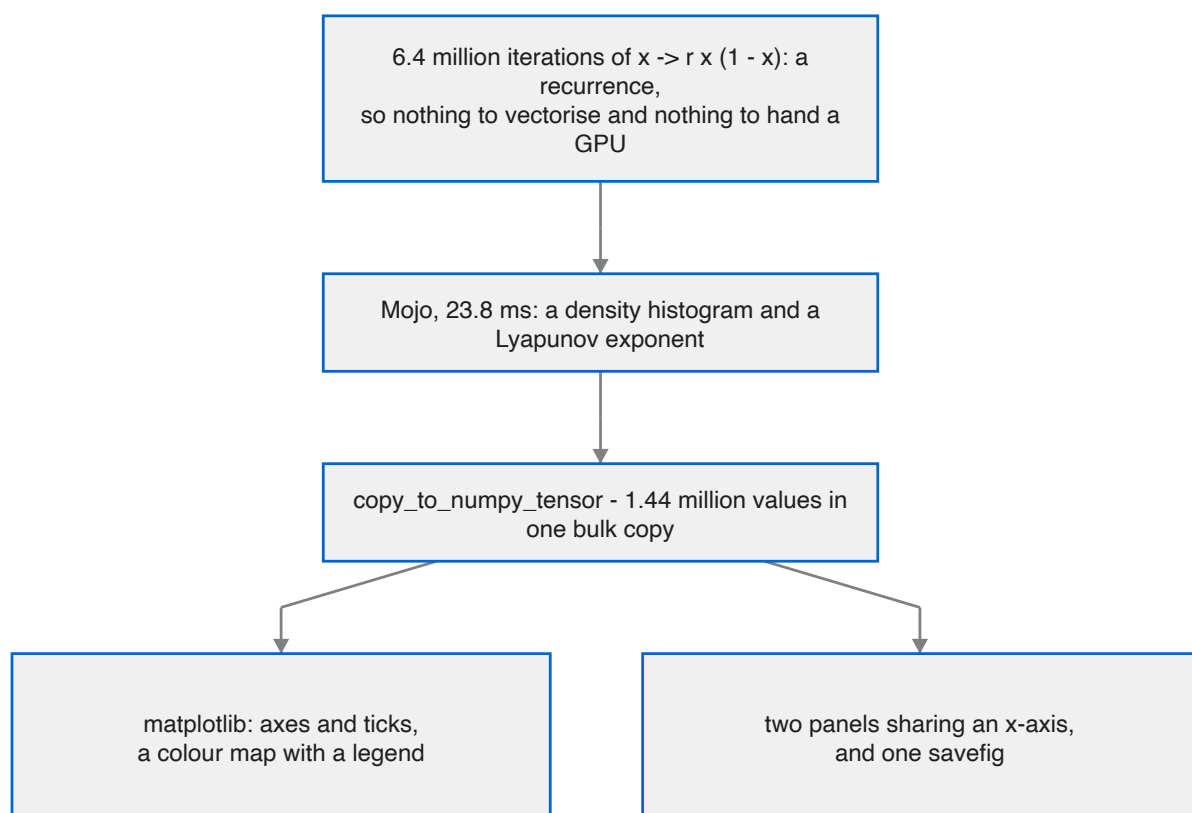
The arithmetic, meanwhile, is a **recurrence** — every iterate depends on the one before it — so there is nothing to vectorise and nothing to hand a GPU. It is simply 6.4 million operations that have to happen in order, which is the one thing CPython is worst at.

So the division is not a preference. Each side gets the half it is actually good at, and the program measures the boundary every time it runs.

The measurement is deliberately unflattering to itself. The CPython side runs the *same* algorithm — histogram write and logarithm included — under `timeit`, which is CPython's best case. The README says what that costs:

On this machine the honest answer is about 20×, not the 100× a badly-set-up comparison would report.

Twenty times is the whole argument. Half a second is tolerable once and intolerable in a loop, and the figure is not the part that costs anything.



1. The map

One line of arithmetic

$$x \rightarrow r x (1 - x)$$

That is the whole model. Take a number between 0 and 1, multiply it by r and by one minus itself, and repeat. It was introduced as a toy model of a population: x is this year's population as a fraction of the maximum, r is the growth rate, and the $(1 - x)$ term is the crowding that stops growth when the population gets large.

It is about as simple as a nonlinear recurrence can be, and it does not behave simply.

What the picture shows

Sweep r from 2.9 to 4.0, and for each value iterate until the orbit settles, then record where it goes:

r	what the orbit does
below 3	settles to one value
3 to ≈ 3.449	oscillates between two
≈ 3.449 to ≈ 3.545	four
then 8, 16, 32...	each doubling in a shorter interval than the last
≈ 3.56995 onward	chaos — the orbit never repeats
≈ 3.8284	a window : period 3, appearing out of chaos

The doubling intervals shrink by a constant ratio — about 4.669 — and that ratio turns out to be *universal*: the same number appears in wholly unrelated systems that period-double. That discovery, by Mitchell Feigenbaum in the 1970s, is what made this equation famous well beyond population biology, and Robert May's 1976 paper is what put it in front of everyone else.

The example's range starts at 2.9, so the first bifurcation at $r = 3$ sits just inside the left edge and the whole cascade is visible.

The windows

The chaotic region is not uniformly chaotic. Inside it are narrow bands where order returns — the orbit collapses back to period 3, or 5, or 6 — and then period-doubles its way back into chaos again, a whole miniature copy of the cascade inside a sliver of r .

The period-3 window near 3.8284 is the famous one, because of a theorem: if a continuous map has an orbit of period three, it has orbits of every period, and its behaviour is chaotic. Period three implies

chaos.

Those windows are the reason the lower panel exists.

The Lyapunov exponent

Looking at the top panel, a window and a chaotic band both look like vertical structure. You can see the difference at period 3, but for a period-6 window in a busy region you cannot.

The lower panel measures it instead:

```
# The Lyapunov exponent: the mean log of |dx'/dx|. Positive means
# nearby orbits separate, which is what chaos is.
var d = r * (1.0 - 2.0 * x)
if d < 0.0:
    d = -d
if d > 1e-12:
    lsum = lsum + log(d)
...
lyap[col] = lsum / Float64(RECORD)
```

The derivative of $r \times (1 - x)$ is $r(1 - 2x)$, which is the local stretching factor: how much a tiny difference between two nearby starting points is multiplied by, at this point on the orbit. Take the log, average it along the orbit, and you have the **exponential rate** at which nearby orbits separate.

- $\lambda < 0$ — differences shrink. Nearby orbits converge; the motion is periodic.
- $\lambda > 0$ — differences grow exponentially. That *is* the definition of chaos: sensitive dependence on initial conditions.

So the bottom panel is not decoration. It is the top panel's claim, made falsifiable:

```
_ = bottom.set_title("\lambda > 0 is chaos; the dips are the windows of order",
                    fontsize=9, loc="left")
```

Every window of order in the top panel has a dip below zero underneath it, at exactly the same r . Two independent computations agreeing is the check.

The `1e-12` guard is worth a glance: at a point where the derivative is exactly zero — a *superstable* orbit, which happens at the centre of every window — the log is negative infinity. Skipping those terms keeps the average finite. It also very slightly biases the exponent upward at the window centres, which is the honest trade for not having an infinity in the array.

Letting the orbit settle

```
comptime TRANSIENT = 2000 # iterations discarded so the orbit settles
comptime RECORD = 2000   # iterations plotted and measured
```

```
# Let the orbit settle before recording anything, or the plot carries
```

```
# a smear of wherever x happened to start.  
var x = 0.5  
for _ in range(TRANSIENT):  
    x = r * x * (1.0 - x)
```

The same idea as the fern's burn-in: the first iterations are on the way to the attractor, not on it. Plot them and every column carries a vertical smear leading down from 0.5, which reads as noise across the whole figure.

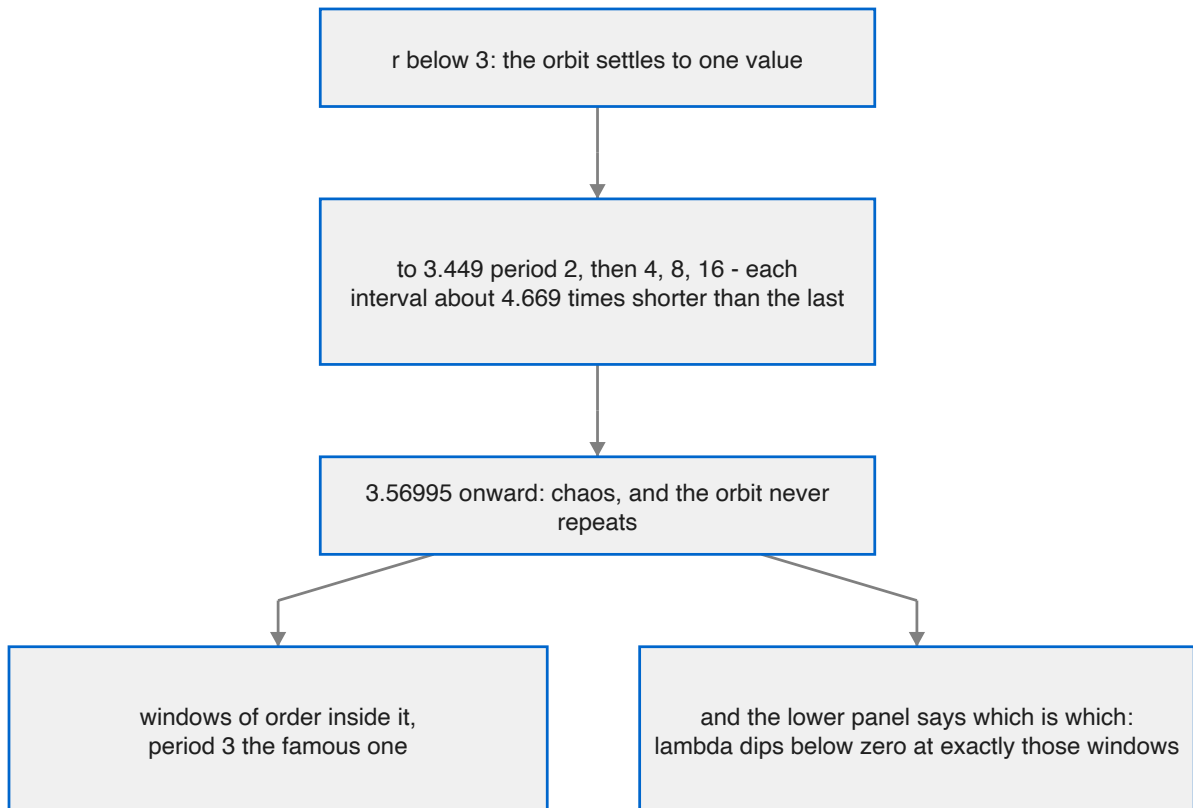
Two thousand each way, per column, over 1,600 columns — 6.4 million iterations, half of them thrown away to make the other half honest.

Why there is nothing to parallelise

```
"""This is the part that has to be fast. Every iteration depends on the one  
before it, so there is nothing to vectorise and nothing to hand a GPU --  
it is simply a lot of arithmetic, done in order."""
```

Within a column, x at step n depends entirely on step $n-1$. No SIMD lane can help; no thread can start early. It is the same structural obstacle the [chaos game](#) has and the same one that keeps alpha-beta off the GPU in [Othello](#).

The columns *are* independent of each other, so 1,600 threads would in principle work. The program does not do it, and at 23.8 ms it does not need to: a dispatch, a buffer and a read-back would cost more than the whole computation. This is the case Othello's alpha-beta measurement makes — **you cannot speed up something that has already finished.**



2. The division of labour

This is the chapter the example exists for, and its argument is stated in the first paragraph of the source:

The thing Python is very good at is not arithmetic. It is everything that surrounds a result: axes with sensible ticks, a colour map with a legend, a log-scaled norm, two panels sharing an x-axis, a label in the right place, and a PNG at the end. That is thousands of decisions somebody has already made well, and reimplementing them is a career rather than an afternoon.

The pair it is half of

The example is explicitly one of two answers, and names the other:

fern/ in this collection writes a PNG by hand — 125 lines to put pixels in a file, with no axes, no ticks, no scale and no legend. It is the right answer when a bitmap is all you want. This example is the other answer.

Compare what the two figures need:

fern/	bifurcation/
a bitmap	two panels sharing an x-axis
no axes	axes with chosen ticks on both
one colour ramp, hand-written	a named colour map with a legend
—	a norm that makes a sparse histogram readable
—	filled regions under a curve, conditionally
125 lines of PNG writer	one savefig

The fern needs none of it, so writing the PNG by hand is 125 readable lines and zero dependencies. The bifurcation diagram needs all of it, and each item is a small pile of decisions — where to put the ticks, how to space them, what to do when a label collides — that somebody has already made well.

Neither is the general answer. **The two sit side by side on purpose**, so that "write it yourself" and "use the library" both appear as reasonable, with the distinguishing question visible: *does this output need presentation, or is a bitmap all you want?*

Why the arithmetic does not go with it

The other half of the argument is that the compute cannot follow the figure across, because it is exactly the shape CPython is worst at:

the map itself is a recurrence — x depends on the x before it — so it cannot be vectorised away, and 6.4 million iterations of it is real work that Mojo does in milliseconds and CPython does in tens of seconds.

The words *vectorised away* are doing real work there. NumPy makes Python fast by moving whole-array operations into C — but that requires the operations to be independent. A recurrence has no such structure: `x[n]` needs `x[n-1]`, so the loop stays in the interpreter, one bytecode dispatch per multiply.

This is the case where "just use NumPy" does not apply, and it is worth being able to recognise.

Measuring it, unflatteringly

The program does not assert the ratio; it measures it, every run, and it takes several deliberate steps to avoid measuring something too favourable.

The same algorithm on both sides.

```
# Deliberately the same work, including the histogram write and the log,
# so the comparison is not flattered by leaving something out.
```

The CPython statement does the transient, the record loop, the bounds check, the histogram increment, the derivative, the absolute value, the guard and the logarithm — every operation the Mojo version does. It would be easy to compare Mojo-with-histogram against Python-without, and it would be meaningless.

CPython's best case, on purpose.

```
"""The same algorithm, in CPython, timed by `timeit`."""
```

From the README:

It is also measured with `timeit`, which runs the loop in a function scope with the collector off — CPython's best case, deliberately.

`timeit` disables the garbage collector and executes in a function scope, where local variable access is an array index rather than a dictionary lookup. Both make CPython meaningfully faster than the same code at module level. The comparison uses them because a benchmark should give the other side its best showing.

Only 24 columns, extrapolated and labelled as such.

```
comptime PY_COLS = 24      # columns CPython is asked to do, for the comparison
```

```
var py_full_s = py_s * Float64(W) / Float64(PY_COLS)
print("  CPython ", one_dp(py_s * 1000.0), "ms  for", PY_COLS,
      "columns  ->", one_dp(py_full_s), "s extrapolated")
```


1,600 columns of CPython would take most of a second every time the example runs, so it does 24 and scales. The output says **"extrapolated"** rather than printing a number that looks measured. The extrapolation is sound because the work per column is identical — but it is still an extrapolation, and the print says so.

And the result is reported to one decimal place:

```
def one_dp(v: Float64) -> String:
    """One decimal place, because six of them is not a measurement."""
```

A ratio printed as `20.238471` claims a precision the experiment does not have.

What the number means

```
grid    1600 columns x 900 rows
work    640000 iterations, each depending on the one before it

Mojo     23.8 ms for all 1600 columns
CPython  8.5 ms for 24 columns -> 0.6 s extrapolated
ratio    20.2 x
```

And the README refuses to inflate it:

On this machine the honest answer is about 20×, not the 100× a badly-set-up comparison would report.

Twenty is a real number and a hundred is a set-up artefact. You can get a hundred by leaving the histogram out, or by timing at module scope, or by comparing against a version that allocates in the loop. The example does none of those, and reports the smaller number.

Then it says why twenty is enough:

Twenty times is the whole argument. Half a second is tolerable once and intolerable in a loop, and the figure is not the part that costs anything.

That is the sharpest sentence in the example. Nobody is arguing 0.6 seconds is unusable. The argument is about the **inner loop of an exploration** — sweeping a parameter, re-running with a finer grid, checking whether a feature is real — where half a second per iteration is the difference between a tool you play with and one you wait on.

And the second clause matters just as much: *the figure is not the part that costs anything*. matplotlib's share of the runtime is irrelevant, so there is no reason to reimplement it. Speed matters where the time is; convenience matters everywhere else.

The failure path is a repair instruction

```
except e:
    print("could not plot:", e)
    print()
    print("matplotlib lives in this project's Python environment:")
    print("  Python menu -> Create or Repair Environment")
    print("  Python menu -> Install Project Dependencies")
```

The likeliest way this example fails on a new machine is a missing matplotlib, which is a setup problem rather than a bug. So the error path prints the two menu items that fix it.

Note also that the comparison is wrapped in its own `try`:

```
except e:
    print("  CPython (comparison skipped:", e, ")")
```

No Python, no comparison — but the Mojo timing still prints and the figure is still attempted. The parts fail independently.

3. Crossing the bridge

Two arrays have to reach Python: a $1,600 \times 900$ histogram and 1,600 exponents. That is 1.44 million values, and how they cross decides whether the bridge is a detail or the bottleneck.

One copy, not 1.4 million

```
# std.python.numpy copies a flat Mojo Span into a real NumPy array. The
# alternative -- appending 1.4 million values across the bridge one at a
# time -- is the slow path, and this is the reason that module exists.
var dens = copy_to_numpy_tensor(density, Coord(H, W))
var lam = copy_to_numpy_array(lyap)
```

Two calls. Each takes a flat Mojo buffer and produces a real NumPy array — one bulk copy, with the shape supplied separately as `Coord(H, W)` so the flat buffer is reinterpreted as a 2-D array rather than reshaped element by element.

The alternative is the one everybody writes first: build a Python list, append in a loop. Each append is a call across the language boundary, a reference count touched, and possibly a list reallocation — and there are 1.4 million of them. That would comfortably cost more than the 23.8 ms of actual computation, which would make the whole argument of [chapter 2](#) collapse: the compute would be fast and the *handover* would be slow.

The README notes that this is the module's reason for existing:

Its own docstring names matplotlib as the case it was written for.

Worth generalising: when a program is fast in one language and presents in another, **the boundary is where the performance goes**. A bulk transfer keeps it a detail; a per-element one makes it the program.

Sending CPython its own source

The comparison needs the same algorithm running in CPython. The example does not ship a `.py` file:

```
def python_seconds(cols: Int) raises -> Float64:
    """The same algorithm, in CPython, timed by `timeit`.

    Sent as source rather than kept in a file beside this one, so the example
    stays a folder with a main.mojo in it. `timeit` takes a statement and a
    setup as strings, which is exactly the shape this needs.
    """
```

Two reasons, and both are good.

The example stays one folder with a main.mojo in it. That is the shape the IDE opens and the shape the build follows, and it is the property that lets every example in the collection be opened and run

without flags. A stray `.py` beside it would work but would make the folder something else.

timeit wants strings anyway. Its API is `timeit(stmt, setup, number=...)` where both are source. Handing it a string is not a workaround — it is the interface.

```
var setup = String("import math") + nl
setup += String("W = ") + String(W) + nl
setup += String("H = ") + String(H) + nl
...
```

The constants are interpolated from the Mojo `comptime` values, so the two implementations cannot drift apart on grid size, transient length or record length. Change `TRANSIENT` in one place and both sides change.

```
return Float64(py=timeit.timeit(stmt, setup, number=1))
```

`number=1` — one run, not the default million. The statement takes 8.5 ms; a million of them would take two and a half hours.

Agg before pyplot

```
# Agg before pyplot: this writes a file and must not need a window.
var mpl = Python.import_module("matplotlib")
_ = mpl.use("Agg")
var plt = Python.import_module("matplotlib.pyplot")
```

matplotlib chooses a rendering backend when `pyplot` is first imported, and the default on a Mac is an interactive one that wants a window and a GUI event loop. This program writes a file.

The ordering is the whole point: `mpl.use("Agg")` must happen **before** `pyplot` is imported, because after that the choice is already made. Get it backwards and the program either opens a window it does not want or fails on a machine with no display — a headless run, a CI box, an SSH session.

Two details worth stealing

The README picks these out, and they are both the kind of thing you learn once by getting them wrong.

The colorbar takes space from both panels

```
# Against both axes, not just the top one. A colorbar takes its space
# from the axes it is given, so attaching it to `top` alone makes the top
# panel narrower than the bottom -- and two panels that share an x-axis
# but do not line up are worse than no colorbar at all.
var cb = fig.colorbar(im, ax=Python.list(top, bottom), pad=0.01)
```

A colorbar is drawn in space *taken from* the axes it is attached to. The colour map belongs to the top panel, so attaching it there is the obvious move — and it shrinks the top panel while leaving the bottom at full width.

The two panels share an x-axis. Misaligning them means an r value in the top panel is at a different screen position from the same r in the bottom, which breaks the one thing the figure is for: reading a window in the top panel and checking that λ dips below zero at the same place.

`ax=[top, bottom]` takes the space from both, and they stay aligned.

PowerNorm, not a log norm

```
# PowerNorm rather than a log norm: most buckets are empty, and a log of
# zero is not a colour.
norm=mcolors.PowerNorm(0.35),
```

The histogram is enormously skewed — a column at a period-2 point puts all 2,000 visits into two buckets, while a chaotic column spreads them across hundreds. Linear colour makes everything but the periodic bands black.

The reflex is a log norm. But most of the 1.44 million buckets are **exactly zero**, and `log(0)` has no colour to map to. matplotlib will either mask those cells or clip them, and either way the empty background stops being background.

`PowerNorm(0.35)` raises the normalised value to the power 0.35 — compressing the high end like a log does, while mapping zero to zero. Same benefit, defined at the bottom of the range.

This is the same problem the [fern](#) solves with `log(h+1)` and `fernwind` solves with `n/(n+K)`. Three density plots, three different compressions, each chosen for what its data actually contains.

The rest of the figure

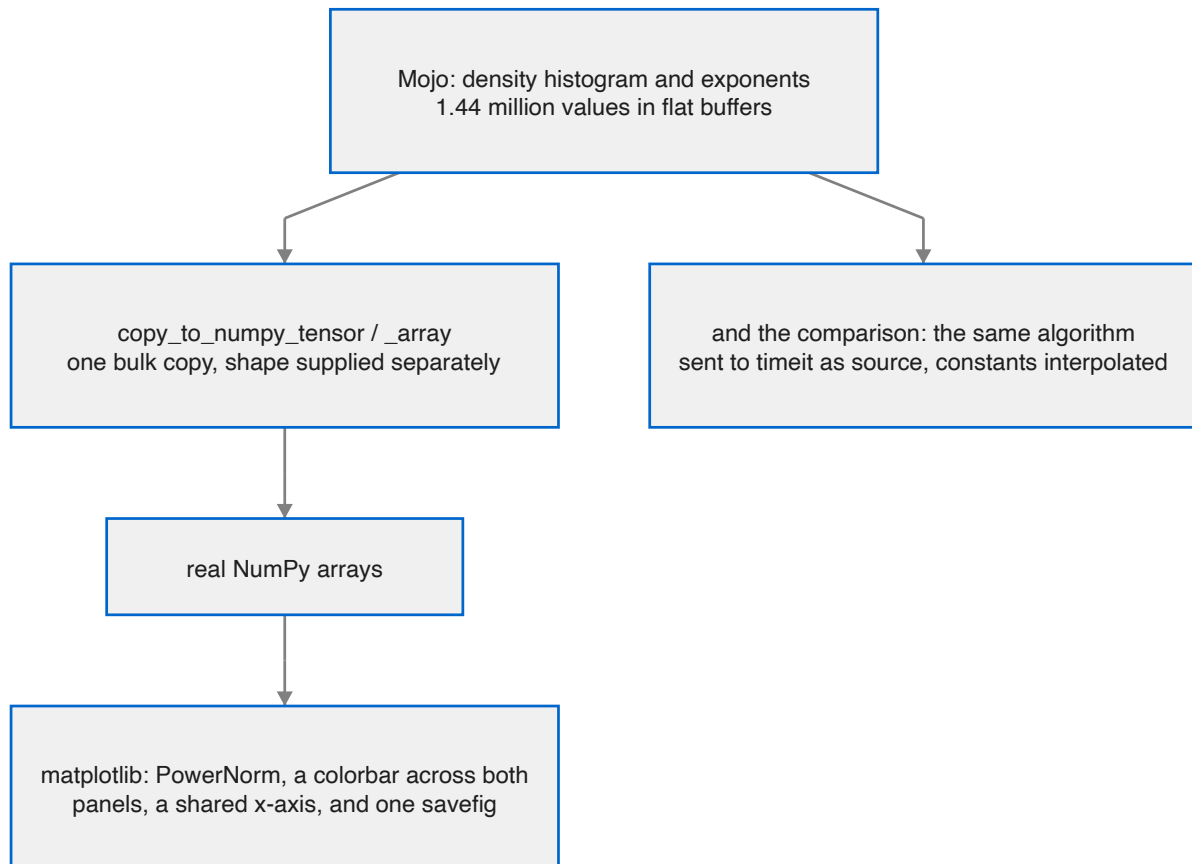
```
var made = plt.subplots(
    2, 1,
    figsize=Python.tuple(12.0, 9.0),
    height_ratios=Python.list(3, 1),
    sharex=True,
    layout="constrained",
)
```

`height_ratios=[3, 1]` — the diagram gets three quarters, the exponent gets one. `sharex=True` locks the axes together. `layout="constrained"` is matplotlib's newer solver, which is what makes the colorbar-and-alignment fix behave.

```
_ = bottom.fill_between(rs, 0.0, lam, where=lam.__gt__(0.0),
                        color="#C2410C", alpha=0.35, linewidth=0)
```

Fill the area under λ **only where it is positive** — so the chaotic regions are shaded and the windows are visibly gaps. `lam.__gt__(0.0)` is the explicit spelling of NumPy's `>` operator, producing a boolean mask array; written this way because the comparison has to reach NumPy rather than be evaluated as a Mojo expression.

Every one of these is a decision somebody already made well, which is the argument the example was written to make.



4. What to understand

Seven things about this example are not obvious from looking at the picture.

1. The division is measured, not asserted

Every run prints the ratio it measured on that machine. Nothing here relies on a number from somewhere else, and the number is checked against the same algorithm on both sides:

```
Mojo      23.8 ms  for all 1600 columns
CPython   8.5 ms  for 24 columns  -> 0.6 s extrapolated
ratio     20.2 x
```

2. Twenty is the honest number, and it is enough

On this machine the honest answer is about 20x, not the 100x a badly-set-up comparison would report.

You could get 100x by leaving the histogram write out, or by timing at module scope instead of in a function, or by comparing against code that allocates in its loop. The example does none of those and reports the smaller figure.

And it says why twenty suffices:

Half a second is tolerable once and intolerable in a loop, and the figure is not the part that costs anything.

Nobody claims 0.6 s is unusable. The claim is about the inner loop of an exploration — sweep, refine, re-run — where half a second per iteration decides whether a tool is interactive.

3. A recurrence is where "just use NumPy" stops working

```
"""Every iteration depends on the one before it, so there is nothing to
vectorise and nothing to hand a GPU -- it is simply a lot of arithmetic,
done in order."""
```

NumPy makes Python fast by pushing whole-array operations into C, which needs the elements to be independent. `x[n]` needs `x[n-1]`, so the loop stays in the interpreter at one bytecode dispatch per multiply.

Recognising this shape is the transferable part. It is the same obstacle the [chaos game](#) has, and the same one that keeps alpha-beta off the GPU in [Othello](#).

4. Not everything that could be parallel should be

The 1,600 columns *are* independent — a GPU could take them. The example does not, and at 23.8 ms it should not: a dispatch, a buffer and a read-back would cost more than the whole computation.

You cannot speed up something that has already finished. Othello measures alpha-beta at 81 μ s and reaches the same conclusion.

5. The bridge is where the performance goes

```
# The alternative -- appending 1.4 million values across the bridge one at a
# time -- is the slow path, and this is the reason that module exists.
var dens = copy_to_numpy_tensor(density, Coord(H, W))
```

Two bulk copies instead of 1.44 million appends. Get this wrong and the compute is fast, the handover is slow, and the whole argument for splitting the work collapses.

6. Density plots need a compression chosen for their data

Three examples in this collection plot density, and none uses the same curve:

fern/	$\log(h+1) / \log(\text{peak}+1)$ — needs a pass to find the peak
fernwind/	$n/(n+3)$ — needs only the pixel; a kernel can afford it
bifurcation/	$\text{PowerNorm}(0.35)$ — because most buckets are exactly zero , and $\log(0)$ is not a colour

The reflex here is a log norm, and it is wrong for this data specifically.

7. Small ordering and layout facts that are easy to get backwards

mpl.use("Agg") before importing pyplot. The backend is chosen at import; afterwards it is too late, and the program either wants a window it should not need or fails on a headless machine.

A colorbar takes space from the axes you give it. Attaching it to the top panel alone narrows that panel while the bottom stays full width — and two panels sharing an x-axis that do not line up defeat the whole figure, because you can no longer read a window in one against the λ dip in the other.

Let the orbit settle. 2,000 discarded iterations per column, or every column carries a smear leading down from $x = 0.5$.

A short list of things that will bite

If you change...	...this happens
the transient to zero	every column smears down from the starting x
PowerNorm to a log norm	the empty buckets stop being background
the colorbar to ax=top	the panels misalign and cannot be read against each other
use("Agg") to after the pyplot import	it wants a window, or fails headless
the bulk copy to a per-element append	the handover costs more than the computation
the CPython statement, without changing the Mojo	the comparison stops comparing the same work
number=1 in timeit	the default is a million runs — about two and a half hours
the 1e-12 guard on the derivative	$\log(0)$ at every superstable orbit centre

Running it

```
cocoamojo --build examples/bifurcation/main.mojo -o /tmp/bifurcation
```

matplotlib lives in the project's own Python environment. Once, from the IDE:

1. Python menu → **Create or Repair Environment**
2. Python menu → **Install Project Dependencies**

Then ⌘R. It prints the ratio it measured, writes `bifurcation.png` beside the project, and opens it.

The other half of this example's argument — writing the PNG by hand, when a bitmap is all you want — is the [fern walkthrough](#).